

Relatório de Laboratório

Laboratório de Experimentação de Software

Curso	Engenharia de Software
Disciplina	Laboratório de Experimentação de Software
Turno / Período	Noite / 6º
Professor(a)	Danilo Maia
Laboratório	Lab01 - Características de Repositórios Populares + Setup do Kanban
Grupo (trio)	Isabella Dias · Leandro Alencar · Luis Henrique
Link do repositório / GitHub Projects	https://github.com/BGLuis/experimentacao-de-software
Data de entrega	27/08/2026

1. Introdução

Com o crescimento massivo do desenvolvimento de software em plataformas de código aberto, o número de "estrelas" no GitHub consolidou-se como o principal indicador de popularidade de um repositório. No entanto, há uma falta de evidências empíricas consolidadas sobre quais características arquiteturais e sociais realmente impulsionam e mantêm o sucesso desses projetos em larga escala. Compreender o perfil desses repositórios importa profundamente para a Engenharia de Software e para a comunidade open-source, pois permite identificar padrões de engajamento, manutenção sustentável e escolhas tecnológicas que diferenciam projetos ativos de sistemas abandonados.

Neste Lab01, investigamos as características fundamentais dos repositórios mais populares do GitHub por meio da mineração de dados via GraphQL, buscando responder a um conjunto de 6 Questões de Pesquisa (RQs). Para cada questão, estabelecemos as seguintes hipóteses informais antes da etapa de validação dos dados:

RQ01: Idade dos Repositórios Populares

Responsável: Luis

Hipótese Informal: Repositórios mais populares tendem a ser maduros e antigos, pois precisam de tempo para construir uma comunidade sólida e acumular milhares de estrelas.

Métrica: Idade do repositório em anos, calculada a partir da data de criação até a data atual)

Validação e Conclusão: A análise dos dados suporta a hipótese. A mediana de idade dos repositórios na amostra foi de 8,72 anos, com média de 8,33 anos e desvio padrão de 4,32 anos. Isso confirma que a grande maioria dos repositórios de sucesso global são projetos bastante estabelecidos e consolidados ao longo de quase uma década.

RQ02: Contribuição Externa (Pull Requests)

Responsável: Luis

Hipótese Informal: Repositórios populares recebem uma quantidade massiva de contribuições externas através de pull requests, refletindo um alto engajamento e descentralização do desenvolvimento.

Métrica: Total de pull requests aceitas (status merged) por repositório.

Validação e Conclusão: Os resultados confirmam fortemente a hipótese. A mediana de PRs aceitas foi de 134, indicando colaboração externa frequente na maioria absoluta dos projetos. A média foi de 1.273,64 com um desvio padrão muito alto (6.215,67),

demonstrando a presença de repositórios gigantescos que concentram volumes extremos de contribuições externas da comunidade.

RQ03: Frequência de Releases

Responsável: Isabella

Hipótese Informal: Repositórios populares tendem a lançar releases com regularidade, mas alguns podem não usar a ferramenta de releases do GitHub, como listas de curadoria e tutoriais.

Métrica: Número total de releases lançadas por cada repositório.

Validação e Conclusão: Na nova amostra dos Top 1.000 repositórios, a mediana de releases subiu consideravelmente para 39,5, indicando uma regularidade de versionamento muito alta entre a elite do GitHub. Contudo, 28% (280 repositórios) não lançaram nenhuma release, 0 releases. Isso reafirma que uma porção enorme dos projetos mais hypados ainda foca em conteúdos passivos (guias, listas) ou não adota releases formais.

RQ04: Frequência de Atualizações (Push)

Responsável: Isabella

Hipótese Informal: Repositórios populares tendem a ser atualizados muito recentemente, pois projetos altamente ativos atraem e mantêm sua base de usuários.

Métrica: Tempo decorrido (em dias) desde a última atualização de código (utilizei o campo pushedAt para maior precisão).

Validação e Conclusão: Os dados dos 1.000 maiores repositórios comprovam uma atividade diária frenética: 40,5% deles receberam atualizações de código nas últimas 24 horas. A mediana global de inatividade é de apenas 3 dias, contra 50 dias da amostra geral antiga. O outlier mais extremo de abandono nesta elite está há 2.448 dias sem um push sequer, mostrando que a fama pode persistir muito além da manutenção ativa.

RQ05: Linguagens Populares

Responsável: Leandro

Hipótese Informal: Repositórios populares tendem a ser escritos em linguagens também populares na comunidade GitHub, conforme a lista de linguagens mais utilizadas do GitHub Octoverse 2025.

Métrica: Linguagem principal de cada repositório. Como a coleta retorna as linguagens ordenadas por tamanho de código, foi considerada como linguagem principal a primeira linguagem listada.

Validação e Conclusão: Na amostra de 1.000 repositórios, 913 (91,30%) possuem uma linguagem identificada e 87 (8,70%) não possuem linguagem informada. Foram encontrados 702 repositórios (70,20% da amostra; 76,89% entre os identificados) em linguagens presentes no GitHub Octoverse 2025, apoiando a hipótese informal. Python (229), TypeScript (174) e JavaScript (110) foram as linguagens mais frequentes. A distribuição apresentou 43 linguagens distintas e 11 categorias raras, com apenas uma ocorrência; esses casos foram mantidos na análise, pois representam diversidade de linguagens e não dados inválidos.

RQ06: Percentual de Issues Fechadas

Responsável: Leandro

Hipótese Informal: Repositórios populares tendem a possuir uma alta proporção de issues fechadas, pois projetos consolidados geralmente possuem manutenção mais ativa.

Métrica: Razão entre issues fechadas e o total de issues: $\text{issues fechadas} / (\text{issues abertas} + \text{issues fechadas})$. Para tornar alto percentual mensurável, foi adotado o critério operacional de pelo menos 75% de issues fechadas.

Validação e Conclusão: Na amostra de 1.000 repositórios, 957 possuíam issues suficientes para calcular a razão; 43 não possuíam issues e foram excluídos apenas desse cálculo. Não foram encontrados valores ausentes ou inválidos. A mediana da razão de fechamento foi 87,62% e 678 repositórios (70,85% dos casos válidos) alcançaram pelo menos 75% de issues fechadas, apoiando a hipótese informal. Foram identificados 40 outliers pela regra de 1,5 vezes o intervalo interquartil, todos na cauda inferior da distribuição; eles foram preservados para representar projetos com padrão de tratamento de issues diferente.

2. Contexto

Este projeto constitui a etapa de visualização e análise de dados no contexto de Mineração de Repositórios de Software da disciplina. Academicamente, este trabalho atua como o ponto de convergência do semestre, onde os dados brutos extraídos e consolidados na etapa de coleta (Lab01) são agora consumidos de forma dinâmica. A partir do arquivo de dados (repositorios_populares_1000.csv) gerado anteriormente, desenvolveu-se um Dashboard interativo que processa as informações em tempo real no próprio navegador (utilizando DuckDB), permitindo uma exploração visual profunda das métricas do projeto.

O objeto de estudo central desta análise são os 1.000 repositórios de código aberto mais populares do GitHub. O objetivo é medir e compreender o estado da arte do desenvolvimento de software open-source por meio da análise da saúde, maturidade e adoção de práticas modernas nesses projetos de alto impacto. Estão sendo medidos aspectos vitais como o ciclo de vida dos projetos (idade e frequência de pushes), engajamento da comunidade (taxa de resolução de Issues e volume de Pull Requests), adoção de Releases e o impacto de novas tendências, como o fenômeno de ferramentas focadas em Inteligência Artificial.

Para fundamentar metodologicamente e interpretar essas métricas, o estudo apoia-se em referências consagradas da Engenharia de Software. A estruturação das Perguntas de Pesquisa (RQs) e dos gráficos obedeceu à essência do paradigma GQM (Goal, Question, Metric), garantindo que a extração de cada métrica responda a um objetivo claro. A avaliação das tecnologias dominantes (RQ05) e das tendências de crescimento em IA dialoga diretamente com os relatórios anuais do GitHub Octoverse, utilizado como índice de referência de popularidade e adoção. Por fim, a análise da frequência de atualizações e da capacidade de resolução de problemas pela comunidade embasa-se indiretamente nos conceitos de performance e saúde de software, adaptando os princípios das métricas DORA para a realidade de projetos mantidos por comunidades de código aberto.

3. Metodologia

3.1 Principais Desafios

A mineração exigiu conciliar profundidade das métricas e limites da API GraphQL do GitHub. Cada repositório demanda consultas para dados de popularidade, atividade, releases, pull requests, issues, linguagens e histórico de commits; por isso, a coleta foi organizada em duas fases e com controle de concorrência por token.

Outro desafio foi a heterogeneidade dos dados. Nem todos os repositórios possuem releases, issues, licença, linguagem ou histórico de commits comparável. Em vez de eliminar automaticamente esses casos, eles foram mantidos na base e excluídos apenas das métricas cujo cálculo não era possível, preservando a representatividade da amostra.

3.2 Tomadas de Decisão

Decisão: A amostra foi fixada nos 1.000 repositórios com maior número de estrelas retornados pela busca do GitHub, para combinar relevância e tempo de processamento viável.

Decisão: A linguagem principal foi definida como a primeira linguagem retornada pela API, ordenada pelo tamanho do código no repositório.

Decisão: A mediana foi usada como medida principal nas métricas assimétricas, pois poucos projetos muito grandes distorcem a média.

Decisão: Para issues, foi adotada a razão issues fechadas / (issues abertas + issues fechadas); repositórios sem issues não entram no denominador da taxa.

Decisão: O dashboard usa DuckDB WASM no navegador, permitindo filtrar e agregar os dados sem transferir a carga analítica para um servidor próprio.

3.3 Etapas

Etapa	Entregas	Responsável(is)
Lab01/Sprint 1	Coleta via GraphQL, definição das RQs e arquivo CSV da amostra.	Luis, Isabella e Leandro
Lab01/Sprint 2	Validação das métricas, scripts de análise e hipóteses.	Luis, Isabella e Leandro
Dashboard	Visualizações interativas, filtros e consultas DuckDB WASM.	Luis, Isabella e Leandro
Relatório final	Síntese metodológica, resultados, gráficos e discussão.	Luis, Isabella e Leandro

O trabalho foi organizado no GitHub Projects, com fluxo Backlog → To Do → Doing → Review → Done. O limite de WIP adotado foi de uma issue em Doing por integrante, evitando concentração de tarefas e facilitando a revisão antes da conclusão.

3.4 Ferramentas

Ferramenta	Uso no estudo
GitHub GraphQL API	Busca dos repositórios e coleta das métricas.
Python	Scripts próprios de mineração, limpeza e análise dos dados.
Pandas	Leitura, transformação e validação do CSV.
React + TypeScript + Vite	Implementação do dashboard interativo.
DuckDB WASM	Consultas SQL, filtros e agregações diretamente no navegador.
GitHub Projects	Planejamento, atribuição e acompanhamento das issues.

3.5 Tabela de Métricas

RQ	Métrica	Definição operacional	Fonte
RQ01	Idade	$(\text{data atual} - \text{criado_em}) / 365,25$	createdAt (GraphQL)
RQ02	PRs aceitas	Quantidade de pull requests no estado MERGED	pullRequests (GraphQL)
RQ03	Releases	Contagem total de releases do repositório	releases (GraphQL)
RQ04	Dias sem atualização	$\text{data atual} - \text{pushedAt}$, em dias	pushedAt (GraphQL)
RQ05	Linguagem principal	Primeira linguagem ordenada pelo tamanho do código	languages (GraphQL)
RQ06	Taxa de issues fechadas	$\frac{\text{issues fechadas}}{(\text{issues abertas} + \text{issues fechadas})} \times 100$	issues (GraphQL)
RQ07	Issues fechadas x estrelas	Taxa de fechamento comparada ao total de estrelas	issues e stargazerCount (GraphQL)
Bônus 01	Wiki e popularidade	Média de estrelas por presença de wiki	hasWikiEnabled e stargazerCount
Bônus 02	Fenômeno IA	Repositórios com tags AI/ML/LLM/GPT por ano	repositoryTopics e createdAt
Bônus 03	Licenças	Frequência de cada licença na amostra	licenseInfo
Bônus 04	Tags e engajamento	Média de estrelas por tag com ao	repositoryTopics e stargazerCount

		menos 20 ocorrências	
Bônus 05	Criação anual	Contagem de repositórios por ano de criação	createdAt

3.6 Inovações Propostas pelo Grupo

Como contribuição adicional, o grupo implementou um dashboard analítico com DuckDB WASM. A solução carrega o CSV no navegador, cria uma tabela local e executa consultas SQL para filtros, estatísticas e gráficos. Essa arquitetura evita o processamento manual de grandes coleções em componentes React e deixa as análises reproduzíveis pela própria interface.

Também foram incluídos gráficos de dispersão renderizados em Canvas, com identificação e acesso ao repositório ao clicar em cada ponto. A escolha torna a exploração da amostra mais direta: o leitor pode sair de uma observação agregada e verificar imediatamente o projeto que a representa.

4. Resultados

4.1 Coleta de Dados

A análise utilizou 1,000 repositórios populares do GitHub. O arquivo possui URL para todos os repositórios da amostra, permitindo rastreabilidade dos pontos apresentados nos gráficos. Foram preservados valores zero e casos sem informação; cada métrica usa somente os registros em que seu cálculo é válido.

4.2 Visualização Gráfica

RQ01 - Idade dos repositórios populares

Pergunta: Sistemas populares são maduros/antigos?

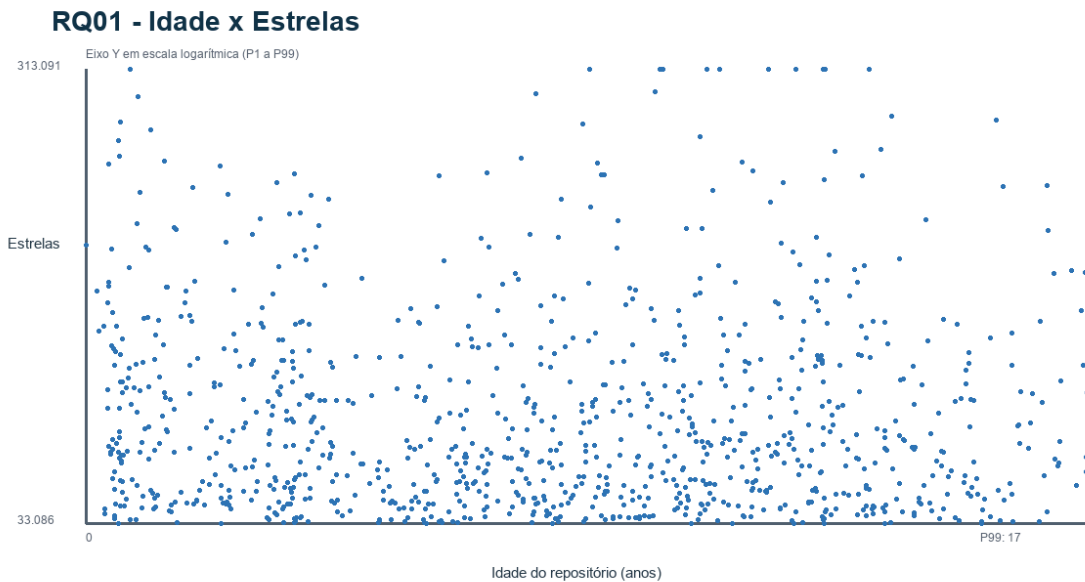


Figura 1 - RQ01 - Idade dos repositórios populares.

A mediana de idade foi de 7,7 anos. A dispersão mostra projetos de diferentes idades, mas a amostra se concentra em repositórios já estabelecidos.

RQ02 - Contribuição externa

Pergunta: Repositórios populares recebem muitas pull requests aceitas?

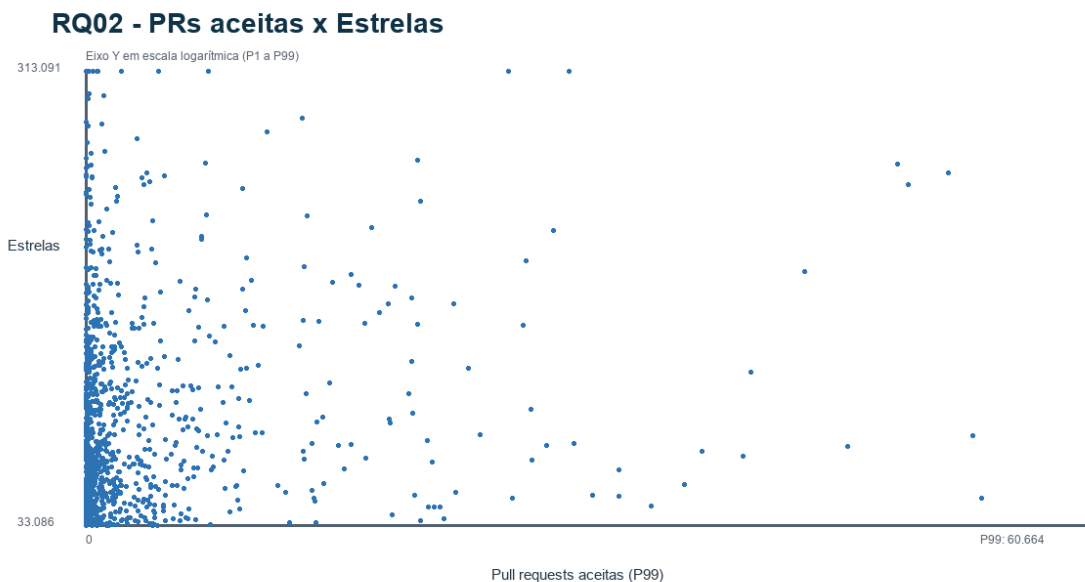


Figura 2 - RQ02 - Contribuição externa.

A mediana de pull requests aceitas foi de 766. A escala logarítmica das estrelas facilita observar os outliers sem esconder a maior parte dos projetos.

RQ03 - Frequência de releases

Pergunta: Repositórios populares publicam releases com regularidade?

RQ03 - Distribuição de Releases

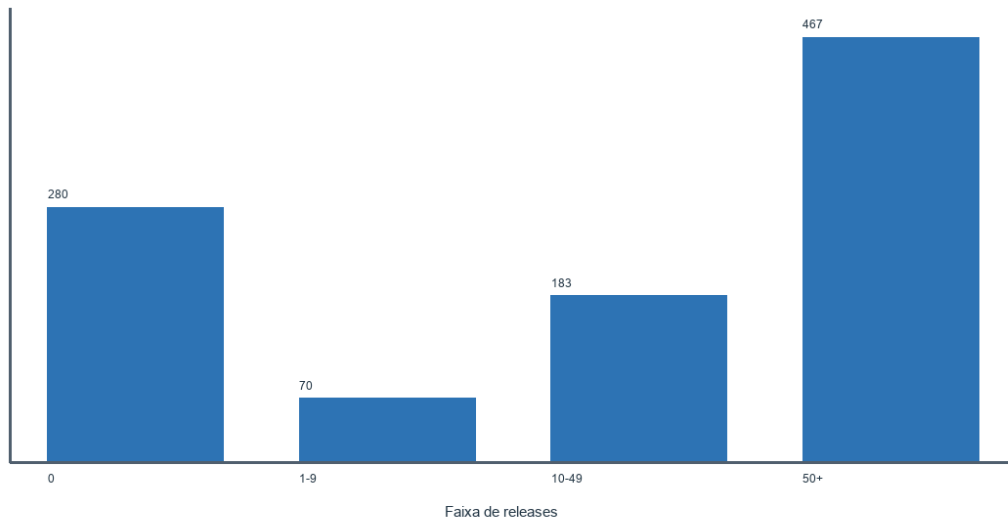


Figura 3 - RQ03 - Frequência de releases.

280 repositórios não possuem releases registradas. O resultado confirma que releases formais não são universais, mesmo entre projetos muito populares.

RQ04 - Frequência de atualizações

Pergunta: Os repositórios populares são atualizados frequentemente?

RQ04 - Tempo desde a última atualização

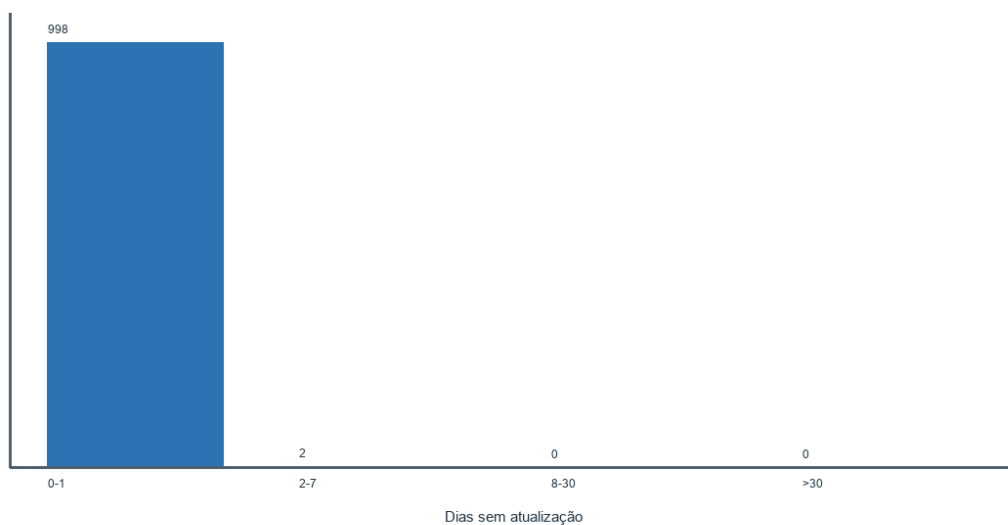


Figura 4 - RQ04 - Frequência de atualizações.

998 repositórios tiveram atualização nos últimos 24 horas; a mediana foi de 0 dias sem atualização.

RQ05 - Linguagens populares

Pergunta: As linguagens dos repositórios populares acompanham as linguagens dominantes?

RQ05 - Linguagens principais mais frequentes

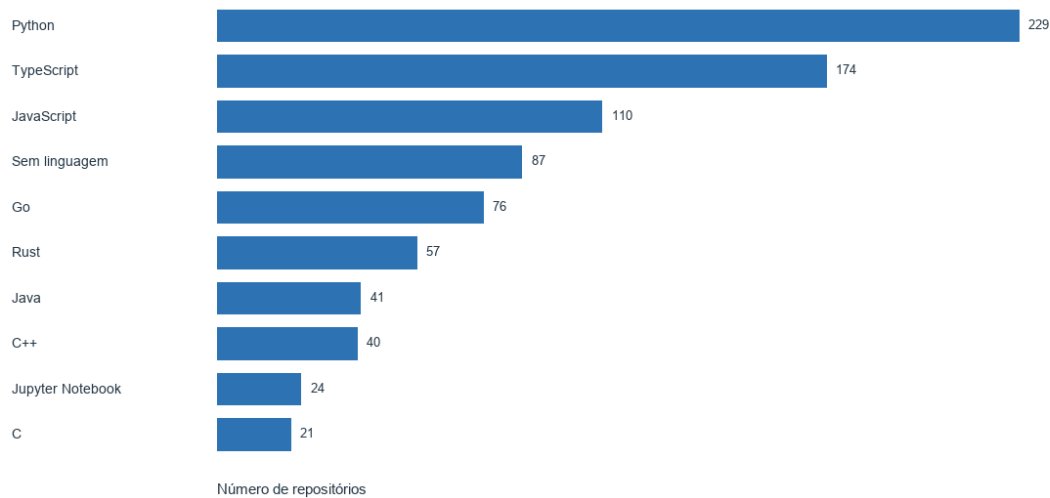


Figura 5 - RQ05 - Linguagens populares.

As linguagens mais frequentes foram Python (229) e TypeScript (174), reforçando a concentração em tecnologias amplamente adotadas.

RQ06 - Percentual de issues fechadas

Pergunta: Projetos populares apresentam alta proporção de issues fechadas?

RQ06 - Fechamento de Issues

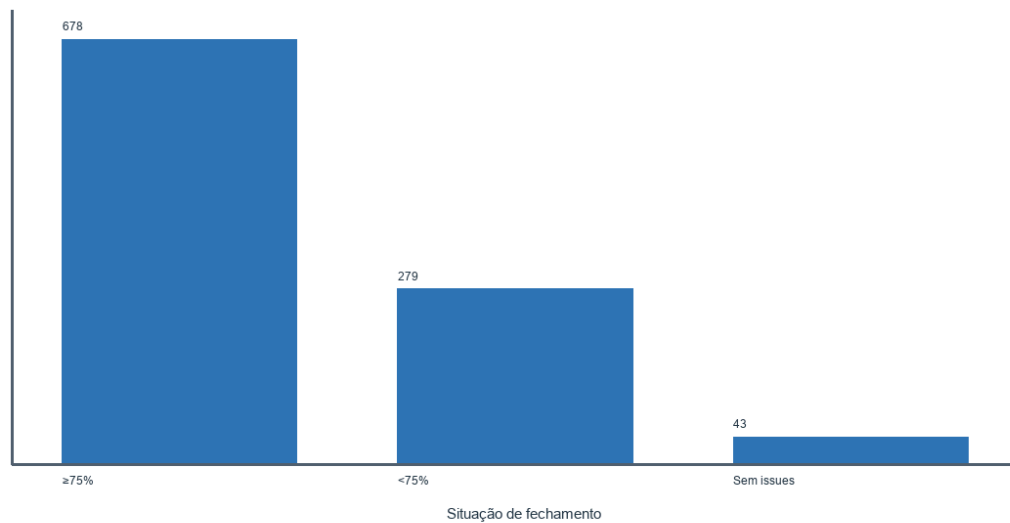


Figura 6 - RQ06 - Percentual de issues fechadas.

Entre os 957 casos válidos, 70.8% atingiram pelo menos 75% de issues fechadas. Isso apoia a hipótese de manutenção ativa na maior parte da amostra.

RQ07 - Issues fechadas e popularidade

Pergunta: A taxa de issues fechadas se relaciona com o número de estrelas?

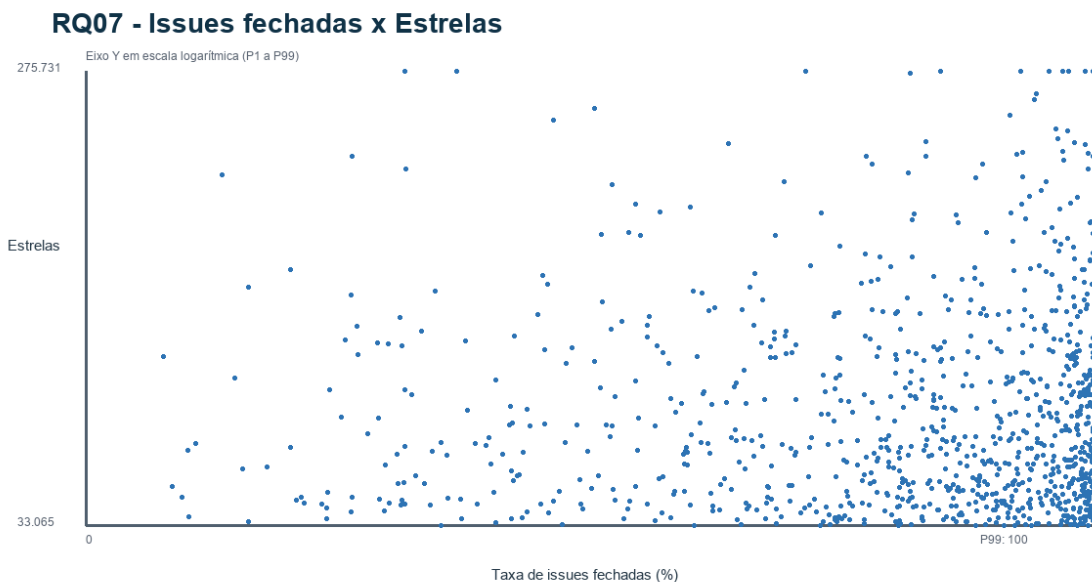


Figura 7 - RQ07 - Issues fechadas e popularidade.

A análise usa 957 repositórios com taxa de fechamento calculável. A dispersão permite observar que popularidade e tratamento de issues são dimensões relacionadas, mas não equivalentes.

Análises Bônus

Bônus 01 - O impacto da Wiki

Pergunta: Projetos com wiki têm perfil de popularidade diferente?

Bônus 01 - Wiki e Popularidade

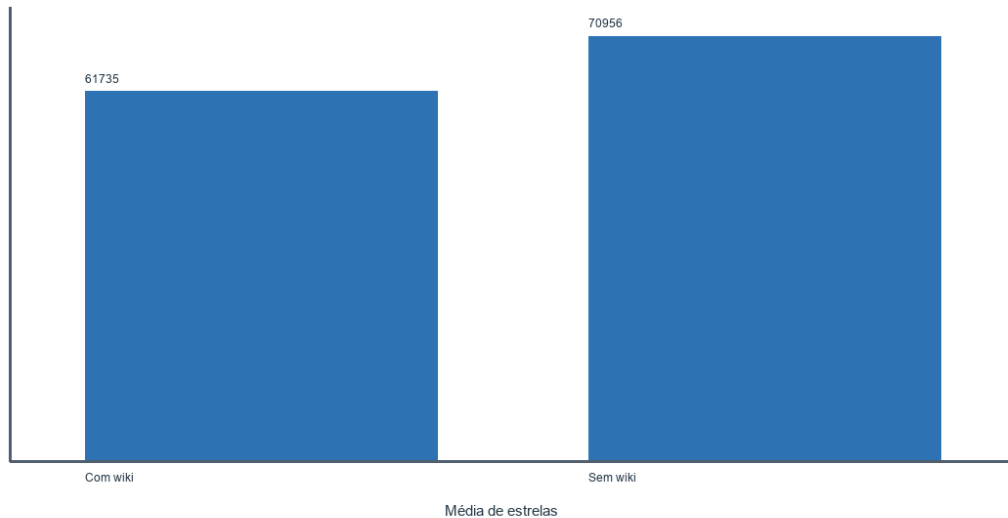


Figura 8 - Bônus 01 - O impacto da Wiki.

A comparação apresenta a média de estrelas entre projetos com e sem wiki; o resultado deve ser interpretado com cautela porque médias são sensíveis a outliers.

Bônus 02 - O fenômeno IA

Pergunta: Como evoluíram os repositórios marcados com temas de IA?

Bônus 02 - Repositórios de IA por ano

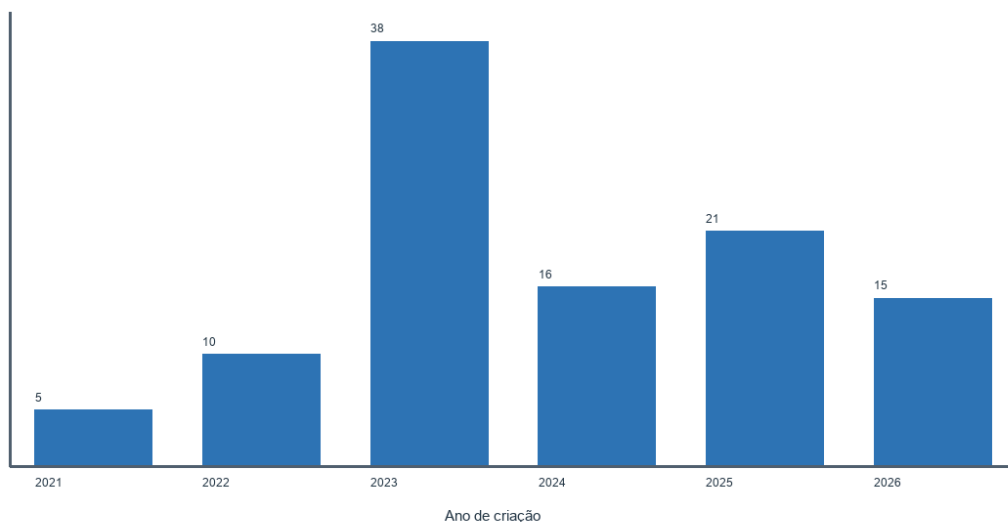


Figura 9 - Bônus 02 - O fenômeno IA.

Foram identificados 116 repositórios com tags associadas a IA, ML, LLM ou GPT. O gráfico evidencia sua distribuição temporal na amostra.

Bônus 03 - Dinheiro, ideologia e licenças

Pergunta: Quais licenças predominam entre os repositórios mais populares?

Bônus 03 - Licenças mais frequentes

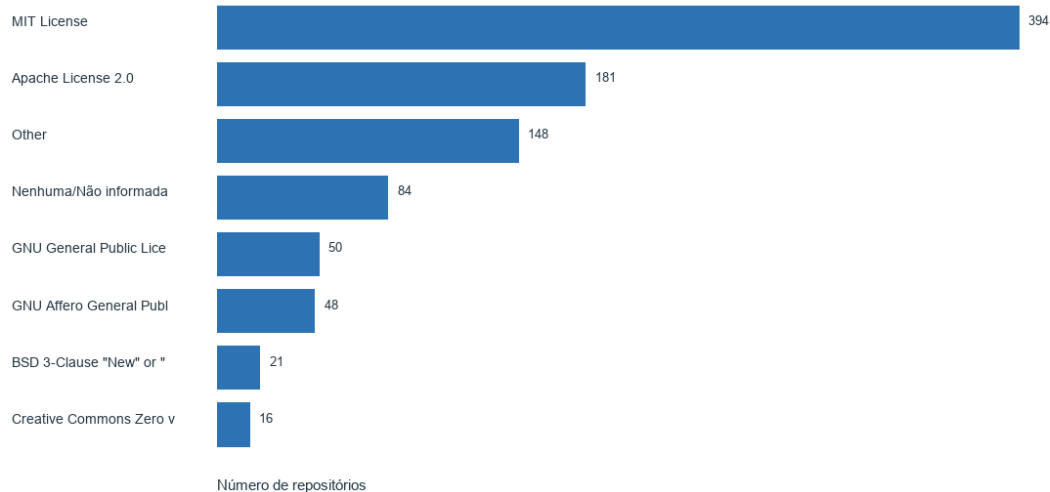


Figura 10 - Bônus 03 - Dinheiro, ideologia e licenças.

A frequência das licenças ajuda a caracterizar a estratégia de abertura e governança adotada pelos projetos mais visíveis.

Bônus 04 - Tendências de tags e engajamento

Pergunta: Quais tags se associam às maiores médias de estrelas?

Bônus 04 - Tags com maior média de estrelas

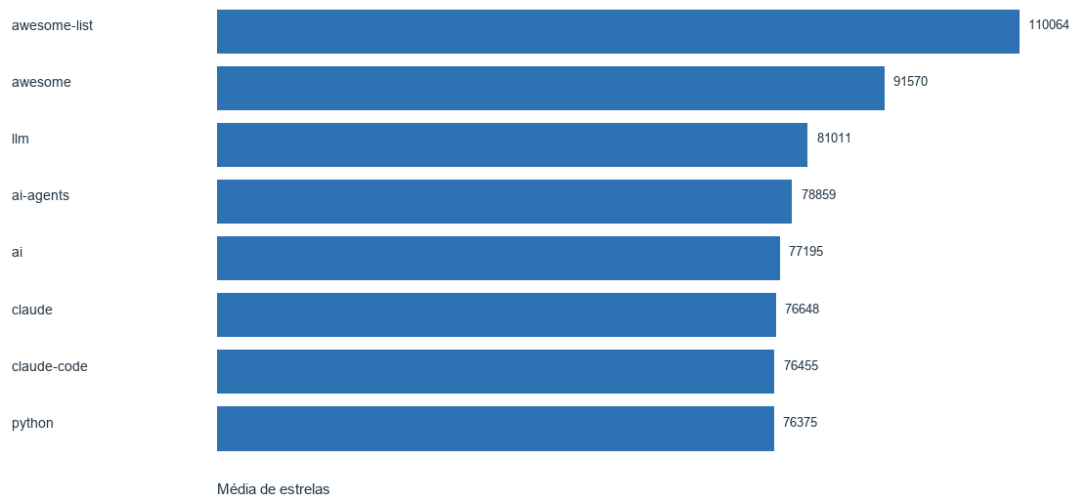


Figura 11 - Bônus 04 - Tendências de tags e engajamento.

Foram consideradas somente tags presentes em pelo menos 20 repositórios, reduzindo a influência de categorias raras.

Bônus 05 - Distribuição por ano

Pergunta: Como os repositórios da amostra se distribuem ao longo dos anos?

Bônus 05 - Repositórios criados por ano

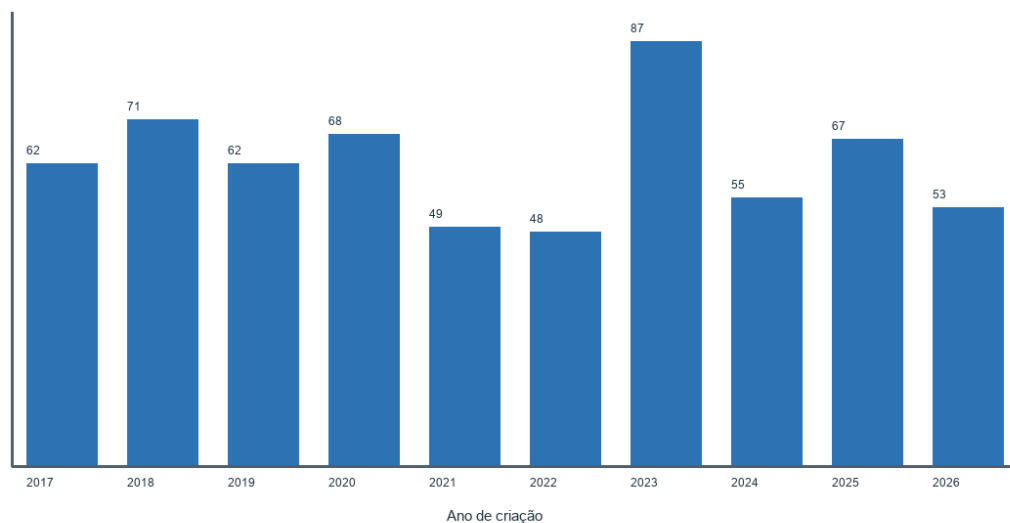


Figura 12 - Bônus 05 - Distribuição por ano.

A distribuição anual complementa a análise de maturidade e mostra os períodos de maior concentração de projetos presentes na amostra.

4.3 Discussão

RQ01 foi apoiada: a idade mediana indica que popularidade tende a acompanhar maturidade, embora existam projetos recentes com grande alcance.

RQ02 foi parcialmente apoiada: há colaboração expressiva, mas a distribuição é muito assimétrica e poucos projetos concentram volumes extremos de pull requests.

RQ03 foi parcialmente apoiada: muitos projetos usam releases, mas a presença de repositórios sem releases demonstra estratégias de distribuição e documentação diferentes.

RQ04 foi apoiada: a concentração de atualizações recentes sugere atividade intensa, porém a cauda de projetos sem push mostra que popularidade e manutenção não são equivalentes.

RQ05 foi apoiada: o ranking é liderado por linguagens de grande adoção na comunidade GitHub, ainda que a amostra mantenha diversidade tecnológica.

RQ06 foi apoiada: a maioria dos casos válidos supera o critério de 75% de fechamento. A principal ameaça à validade é que uma issue fechada não informa, por si só, qualidade ou rapidez da resolução.

RQ07 e as análises bônus ampliam a interpretação das RQs principais ao relacionar popularidade com gestão de issues, documentação, temas de IA, licenças, tags e período de criação. Esses resultados são exploratórios e não estabelecem causalidade.

5. Conclusão

Os 1.000 repositórios analisados mostram um ecossistema marcado por maturidade, atividade e forte participação comunitária. As métricas não indicam um único perfil de sucesso: releases, pull requests e atualizações variam bastante entre os projetos, enquanto linguagens consolidadas permanecem dominantes.

A principal limitação é a amostra transversal: ela retrata os repositórios mais estrelados em um ponto no tempo e não mede causalidade. Como trabalho futuro, o grupo pode comparar coletas periódicas e medir a evolução das métricas. O dashboard com DuckDB WASM e acesso direto aos repositórios oferece uma base prática para essa continuidade.

6. Referências

GitHub Docs. GraphQL API. Disponível em: <https://docs.github.com/en/graphql>

GitHub Octoverse. "A new developer joins GitHub every second as AI leads TypeScript to #1". 2025. Disponível em: <https://github.blog/news-insights/octoverse/octoverse-a-new-developer-joins-github-every-second-as-ai-leads-typescript-to-1/>

DuckDB. DuckDB-Wasm. Disponível em:

<https://duckdb.org/docs/stable/clients/wasm/overview.html>

Vídeo de referência. Disponível em: <https://www.youtube.com/shorts/YwnaeO95AN8>